

PHASE 6

Testing Results and Discussion

Chapters 9–10: test strategy, test cases, results and discussion

FITBUDDY • AI FITNESS PLAN GENERATOR • PROJECT REPORT

Chapter 9: Testing

9.1 Testing Strategy

Testing is planned at several levels so that faults are found close to where they are introduced.

- Unit testing checks individual functions in isolation, such as schema validation, JSON parsing of model output and helper functions. The Gemini client is replaced by a stub so that tests are repeatable and do not consume API quota.
- Integration testing checks that the route, generator module and database work together, for example that a valid request results in stored user and plan records.
- API testing exercises endpoints through FastAPI's test client or the Swagger UI, checking status codes and response content for valid and invalid requests.
- Database testing checks that records are created, related and retrieved correctly, that required fields are enforced and that the original plan is unchanged after a revision.
- UI testing checks form behaviour, page rendering and responsiveness on desktop and mobile widths, in more than one browser.
- Error-handling testing simulates an unavailable Gemini service, a malformed model response and invalid input, and checks that the user receives a clear message and that no partial data is stored.

Tests that call the live Gemini API are inherently variable, so they check structure (seven days, required fields) rather than exact wording.

9.2 Test Cases

Table 9.1 lists the planned test cases. No execution evidence was available when this report was prepared, so the actual results are left as placeholders and every status is marked To Be Verified. These columns should be completed after the tests are run.

Table 9.1: Planned Test Cases

ID	Test Scenario	Input	Expected Result	Actual Result	Status
T1	Valid user input	All fields valid	Input accepted; generation starts	[Insert Actual Test Result]	To Be Verified
T2	Missing name	Name empty	Validation error; no record created	[Insert Actual Test Result]	To Be Verified
T3	Invalid age	Age = text or out of range	Validation error message	[Insert Actual Test Result]	To Be Verified
T4	Invalid weight	Weight = 0 or negative	Validation error message	[Insert Actual Test Result]	To Be Verified
T5	Missing goal	Goal empty	Validation error message	[Insert Actual Test Result]	To Be Verified
T6	Missing intensity	Intensity empty	Validation error message	[Insert Actual Test Result]	To Be Verified
T7	Valid workout generation	Valid profile	Seven-day plan with required fields shown	[Insert Actual Test Result]	To Be Verified
T8	Gemini API unavailable	Invalid key or no network	Clear error message; no partial data stored	[Insert Actual Test Result]	To Be Verified
T9	Nutrition generation	Valid profile	Short nutrition/recovery tip shown	[Insert Actual Test Result]	To Be Verified
T10	Feedback submission	Feedback text for an existing plan	Feedback accepted and stored	[Insert Actual Test Result]	To Be Verified
T11	Revised plan generation	Feedback such as "shorter sessions"	Revised seven-day plan shown	[Insert Actual Test Result]	To Be Verified
T12	Original plan preservation	Submit feedback, then inspect the plan record	Original plan text unchanged	[Insert Actual Test Result]	To Be Verified
T13	Database storage	Valid request	User, plan and tip rows present	[Insert Actual Test Result]	To Be Verified
T14	Admin user retrieval	Authorised dashboard request	Users, plans, feedback and revisions listed	[Insert Actual Test Result]	To Be Verified
T15	Invalid user ID	Feedback for a non-existent user	Clear error message; nothing stored	[Insert Actual Test Result]	To Be Verified
T16	API documentation	Open /docs	Swagger UI lists all endpoints	[Insert Actual Test Result]	To Be Verified
T17	Responsive UI	Desktop, tablet and phone widths	Layout adapts; no horizontal page scroll	[Insert Actual Test Result]	To Be Verified
T18	Multiple users	Two or more distinct profiles	Records stored separately; no data mixing	[Insert Actual Test Result]	To Be Verified

Chapter 10: Results and Discussion

This chapter distinguishes between behaviour the design is intended to produce and behaviour that has been observed. Because no execution evidence was available when this report was prepared, the sections below describe expected system behaviour. Observed results are left as placeholders to be completed from actual runs. No performance figures are given, since none were measured.

10.1 Expected System Behaviour

- Personalised workout generation. For a valid profile, the system is expected to return a seven-day plan whose exercise selection and duration reflect the stated goal and intensity. Different profiles should produce visibly different plans.

- Nutrition recommendation. A short, general tip covering hydration, eating and recovery is expected alongside the plan.
- Feedback-based adaptation. After feedback such as "sessions are too long", the revised plan is expected to shorten sessions while keeping seven days and the same structure.
- Database persistence. Users, plans, tips, feedback and revisions are expected to remain available after the application is restarted, because they are stored in the SQLite file.
- Administrative monitoring. An authorised administrator is expected to be able to see all stored records in one view.
- API accessibility. The endpoints and request models are expected to be discoverable and testable at /docs.
- User experience. The interface is expected to be readable on desktop and mobile, with a progress indication during AI processing.

10.2 Observed Results

[Insert observed results here: example generated plan, example revision and the change between them, screenshots of stored records, and any measured response times. [Insert Actual Test Result]]

10.3 Discussion

Three design decisions are expected to matter most in practice. First, validating input before calling the model avoids wasted requests and keeps malformed data out of prompts. Second, requesting structured JSON allows the interface to render plans consistently, although the parsing step must tolerate occasional malformed output. Third, storing revisions separately supports comparison and review of AI behaviour.

Response time is expected to be dominated by the Gemini calls rather than by the web or database layers, and it varies with network conditions and service load. Generated content quality varies between runs and should be reviewed by a person with fitness knowledge before the system is used with real users. These points are stated as expectations and should be confirmed or revised once measurements are available.